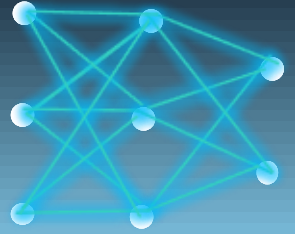




Deep Learning

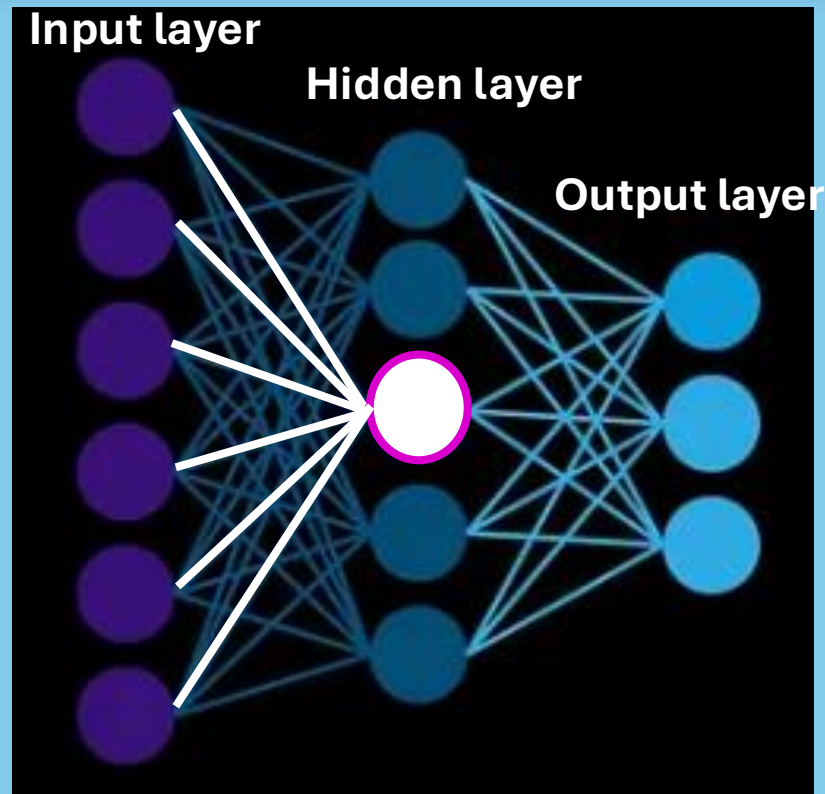
Dr. Shay Maymon

Deep Feedforward Networks / Feedforward Neural Networks / Multilayer Perceptrons (MLPs)



Training Examples

$$\{(x_n, y_n \approx f^*(x_n))\}$$

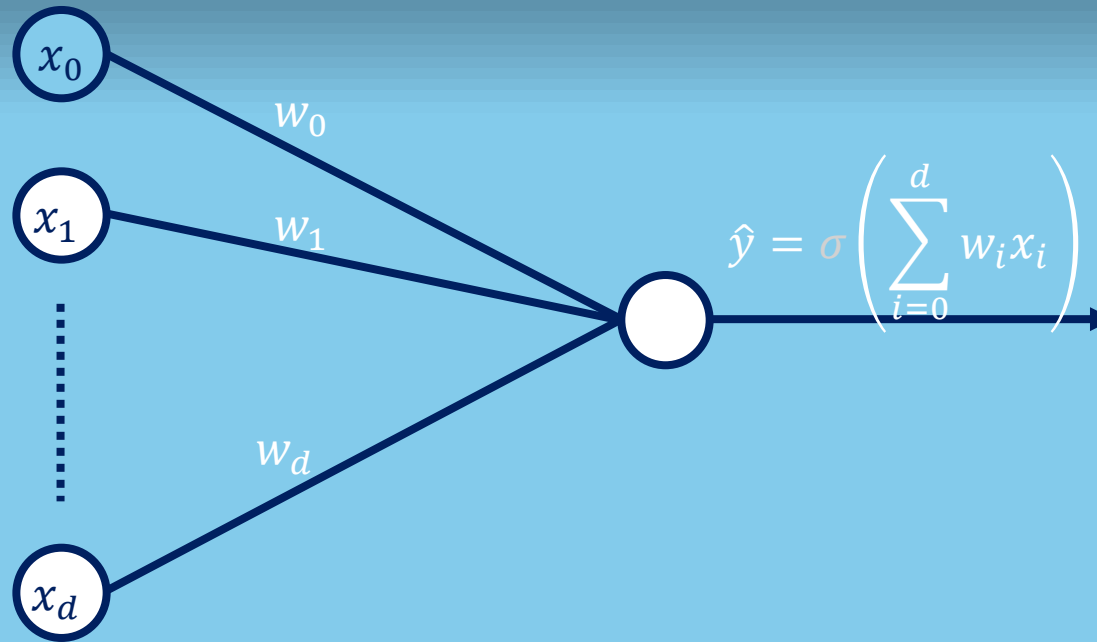
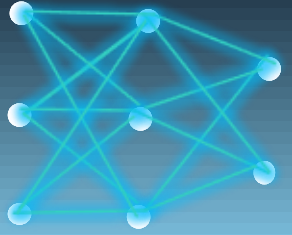


$$\hat{y} = f(x, \theta) \approx f^*(x)$$

The training data
specify directly what
the output layer
must do at each
point x

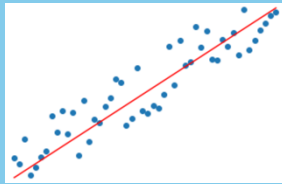
The learning algorithm must decide how to
use the hidden layers to best approximate f^*

Linear Models



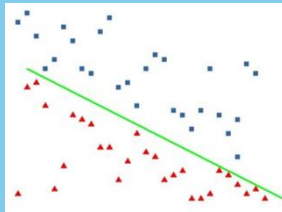
Linear Regression:

$$\frac{1}{N} \sum_{n=1}^N (y_n - \mathbf{x}_n^T \mathbf{w})^2$$

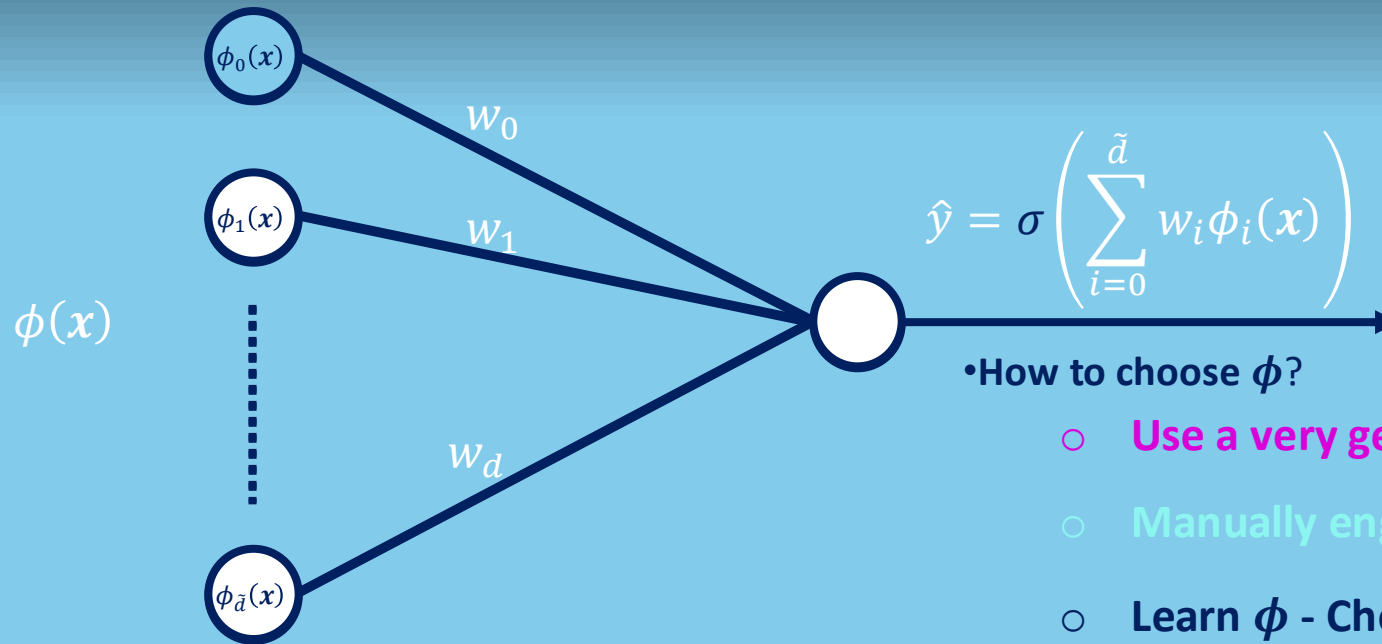
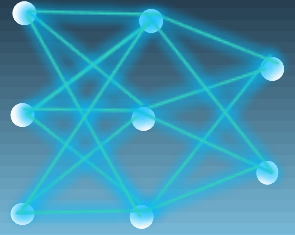


Logistic Regression:

$$\frac{1}{N} \sum_{n=1}^N \log(1 + e^{-y_n \mathbf{x}_n^T \mathbf{w}})$$



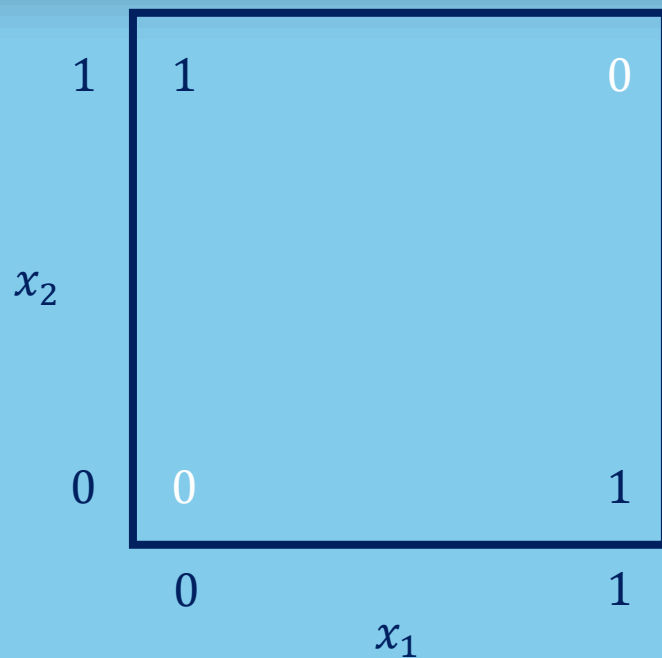
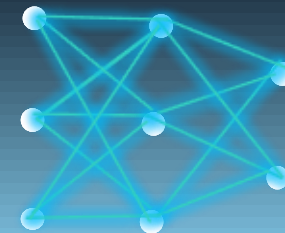
Extending Linear Models to Nonlinear Functions



•How to choose ϕ ?

- Use a very generic ϕ
- Manually engineer ϕ
- Learn ϕ - Choose basis functions that themselves have learnable parameters

Example



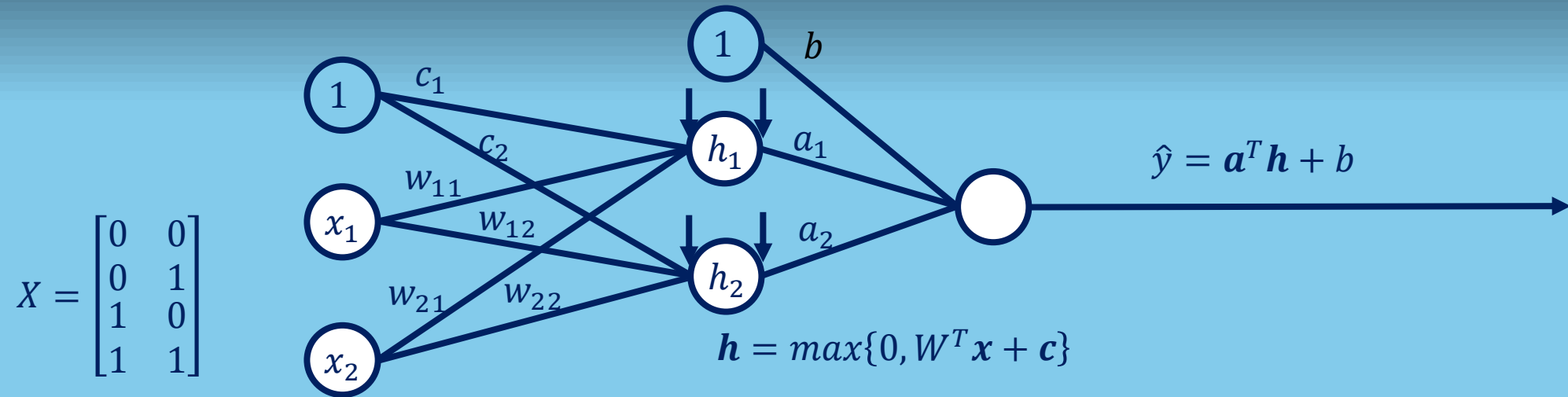
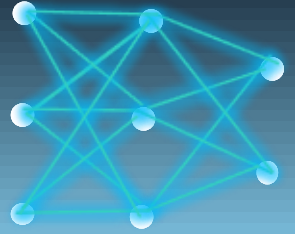
$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Linear Model:

$$f(x; w, b) = w^T x + b$$

A linear model cannot represent the solution!

Example: Non-Linear Model



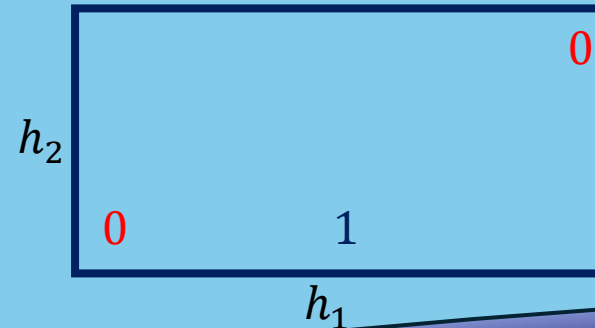
$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

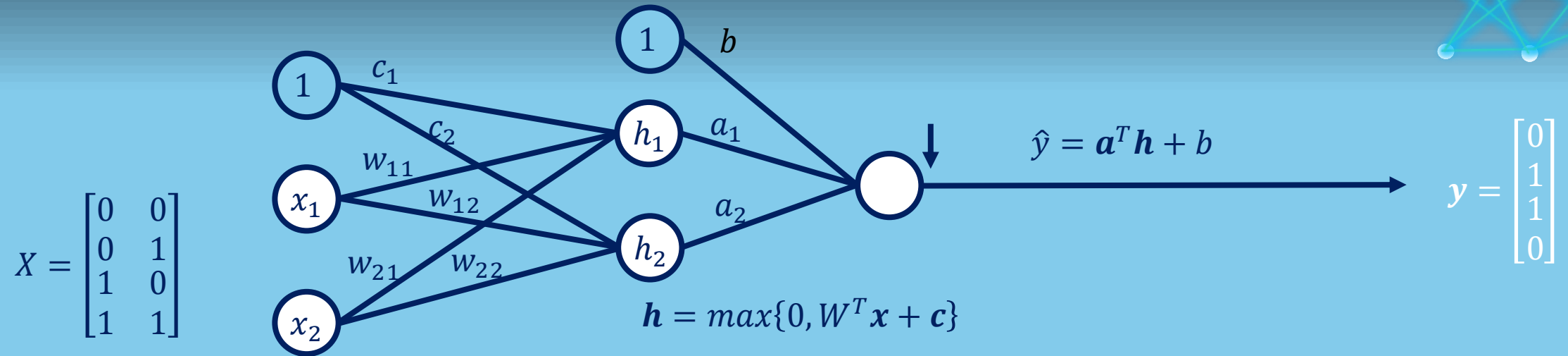
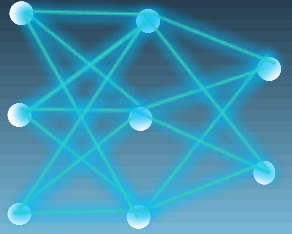
$$W^T x = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \end{bmatrix} \right\}$$

$$W^T x + c = \left\{ \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right\}$$

$$h = \max\{0, W^T x + c\} = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right\}$$



Example: Non-Linear Model

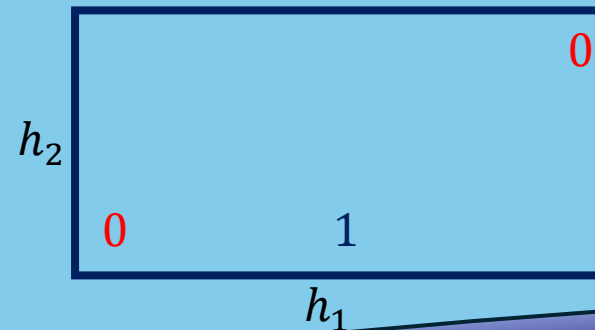


$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

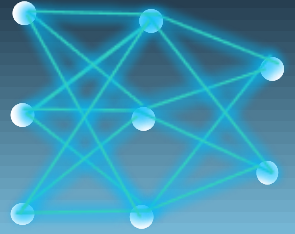
$$a = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad b = 0$$

$$\hat{y} = a^T h + b = \{0, 1, 1, 0\}$$

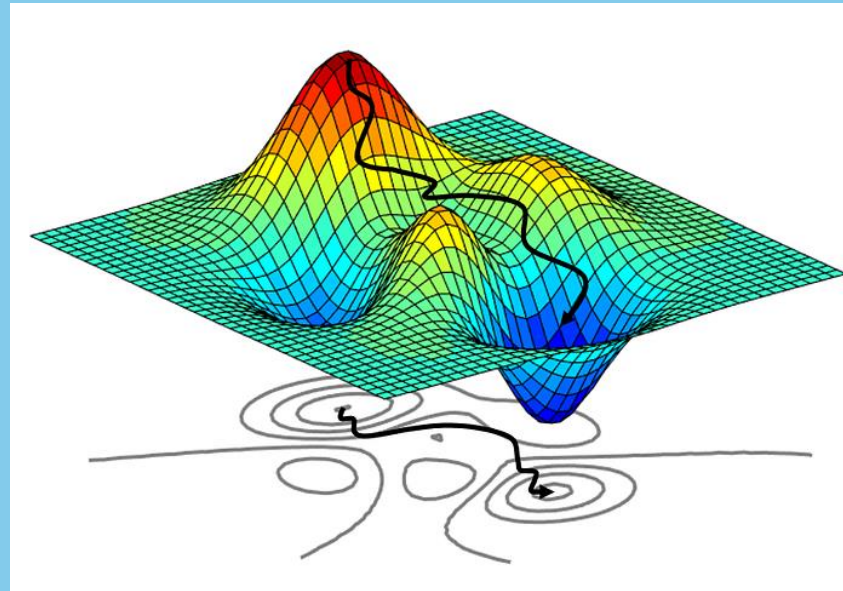
$$h = \max\{0, W^T x + c\} = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right\}$$

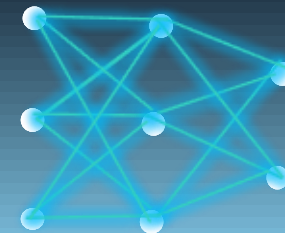


Gradient-based Learning



- **Neural networks** are typically **trained** using **iterative, gradient-based optimizers** that merely drive the cost function to a very low value

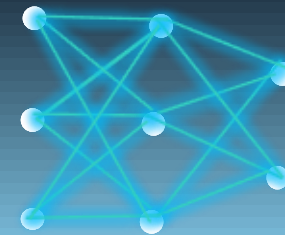




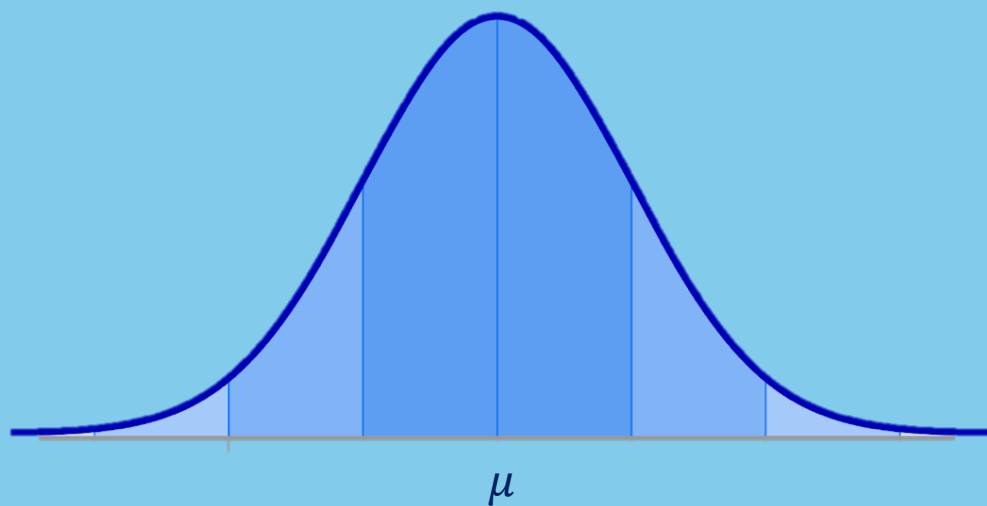
Output units & Cost function

- To apply gradient-based learning **we must choose**
 - **how to represent the output of the model**
 - **cost function**
- The **role of the output layer** is to **provide some additional transformation from the features to complete the task** that the network must perform
- The choice of **cost function** is **tightly coupled** with the choice of **output unit**

Estimation

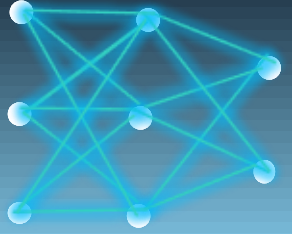


$$f_y(y; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y-\mu)^2}$$



Given a realization of y how would you estimate μ ?

Maximum Likelihood Estimation

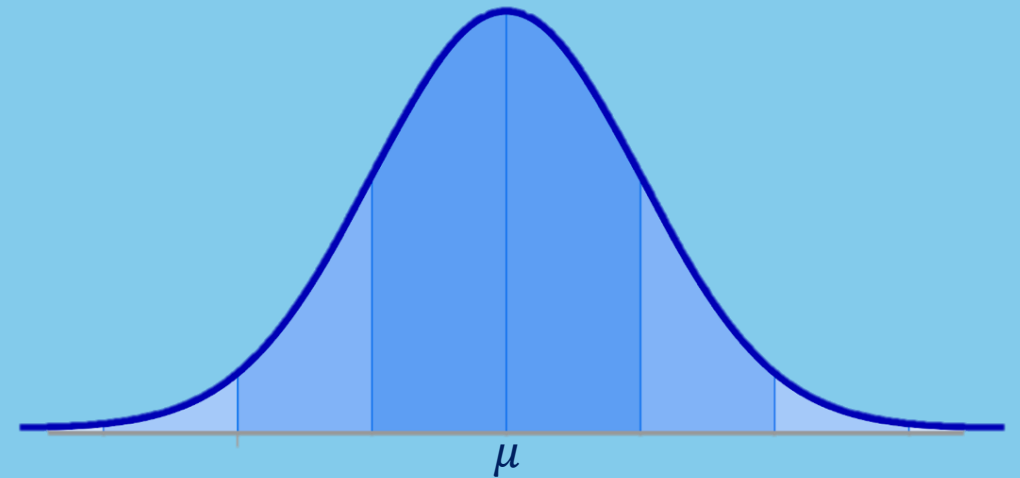


- Maximum likelihood estimators have proven **popular** in many applications
- In many cases they have **good asymptotic properties**

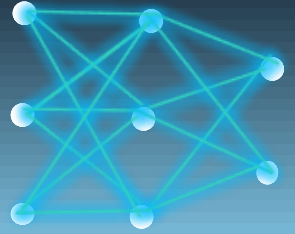
$$L(\boldsymbol{\theta}) = f_{\mathbf{y}}(\mathbf{y}; \boldsymbol{\theta})$$

$$\hat{\boldsymbol{\theta}}_{ML} = \operatorname{argmax}_{\boldsymbol{\theta}} f_{\mathbf{y}}(\mathbf{y}; \boldsymbol{\theta})$$

$$\hat{\boldsymbol{\theta}}_{ML} = \operatorname{argmax}_{\boldsymbol{\theta}} \log f_{\mathbf{y}}(\mathbf{y}; \boldsymbol{\theta})$$



Cost functions

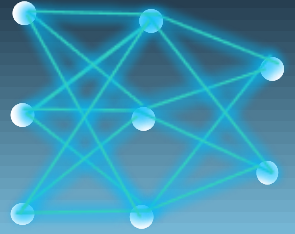


- **Most modern neural networks** are **trained** using **maximum likelihood**
- In most cases, **our parametric model** defines a distribution $p(y|x; \theta)$ and we simply use the principle of **maximum likelihood**
- This means that the **cost function** is simply the **negative log-likelihood**, equivalently described as the **cross-entropy** between the **training data** and the **model distribution**

Advantages?

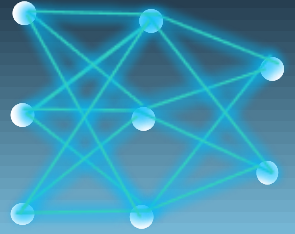
An **advantage** of this approach of **deriving the cost function from maximum likelihood** is that it removes the **burden of designing cost functions** for each model

Cost functions



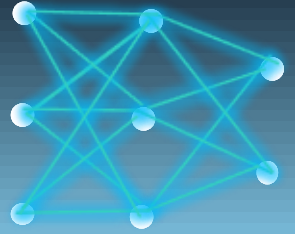
- Sometimes, we take a **simpler approach**, where **rather than predicting a complete probability distribution over y** , we merely **predict some statistic of y conditioned on x**

Cost functions



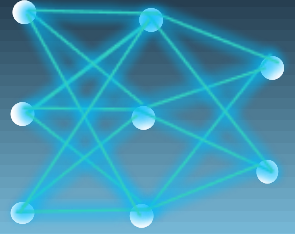
- The **total cost function** used to train a neural network will often **combine** one of the **primary cost functions** described here with a **regularization term**

The Gradient of the Cost functions



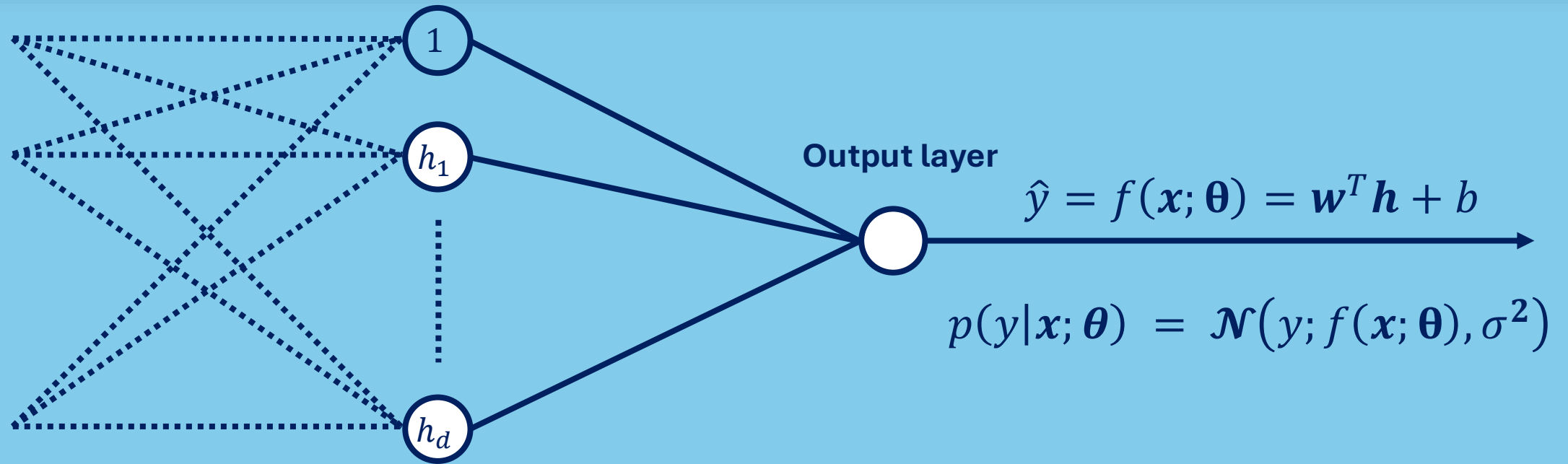
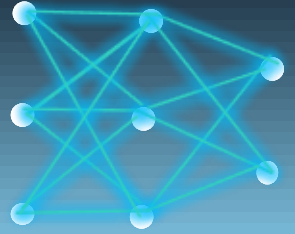
- The **gradient of the cost function** must be **large** to serve as a **good guide** for the **learning algorithm**
- Functions that **saturate** (become very flat) **undermine this objective** because they make the gradient become very small. In many cases this happens because the **activation functions** used to produce the **output of the hidden units**, or the **output units saturate**
- The **negative log-likelihood helps to avoid this problem** for many models. Many **output units** involve an **exp function** that can **saturate** when its argument is very negative. The **log function** in the **negative log-likelihood cost function** can **undo** the **exp** of some output units

MSE& MAE as Cost functions

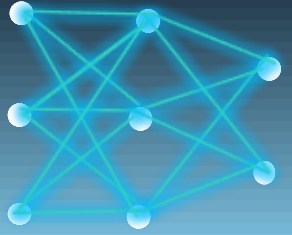


- **Mean squared error** and **mean absolute error** often lead to **poor results** when used with **gradient-based optimization**
- Some **output units that saturate** produce **very small gradients** when combined with these cost functions

Linear units for Gaussian Output distribution



Linear Units for Gaussian Output distribution



Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.

$$p(y_n | \mathbf{x}_n; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2}$$

$$L(\boldsymbol{\theta}) = p(\mathbf{y} | \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{\theta}) = \prod_{n=1}^N p(y_n | \mathbf{x}_n; \boldsymbol{\theta}) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2}$$

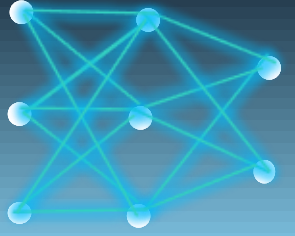
$$\log L(\boldsymbol{\theta}) = \sum_{n=1}^N \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2} \right) = \sum_{n=1}^N -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

$$\log L(\boldsymbol{\theta}) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

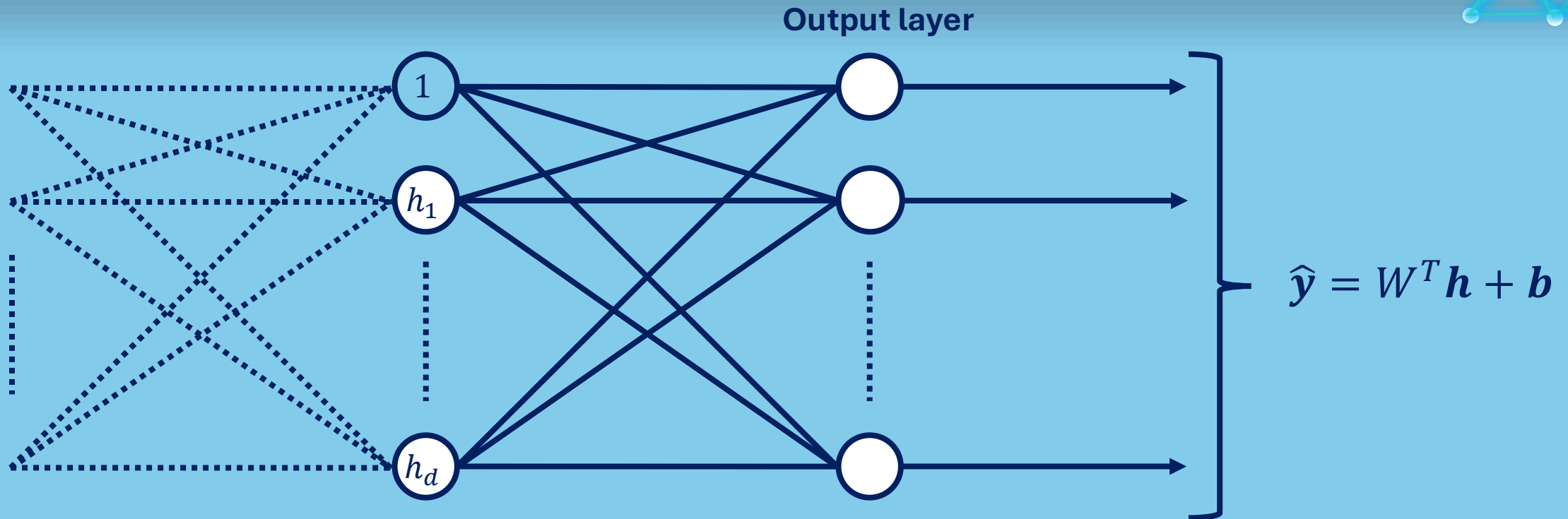
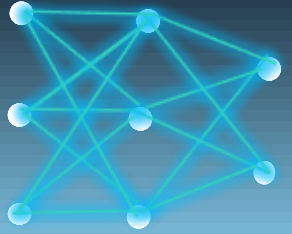
$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^N (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

Linear units for Gaussian Output distribution



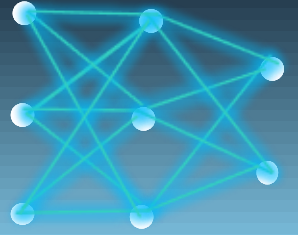
Because Linear units do not saturate, they pose little difficulty for gradient-based optimization algorithms and may be used with a wide variety of optimization algorithms

Linear units for Gaussian Output distribution



Such a network **can approximate** any **continuous function** from x to y

Linear Units for Gaussian Output distribution



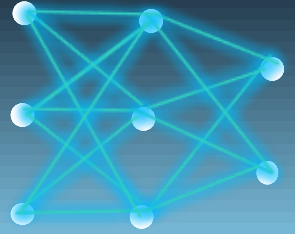
Homework

$$\hat{y} = f(x; \theta) = W^T h + b$$

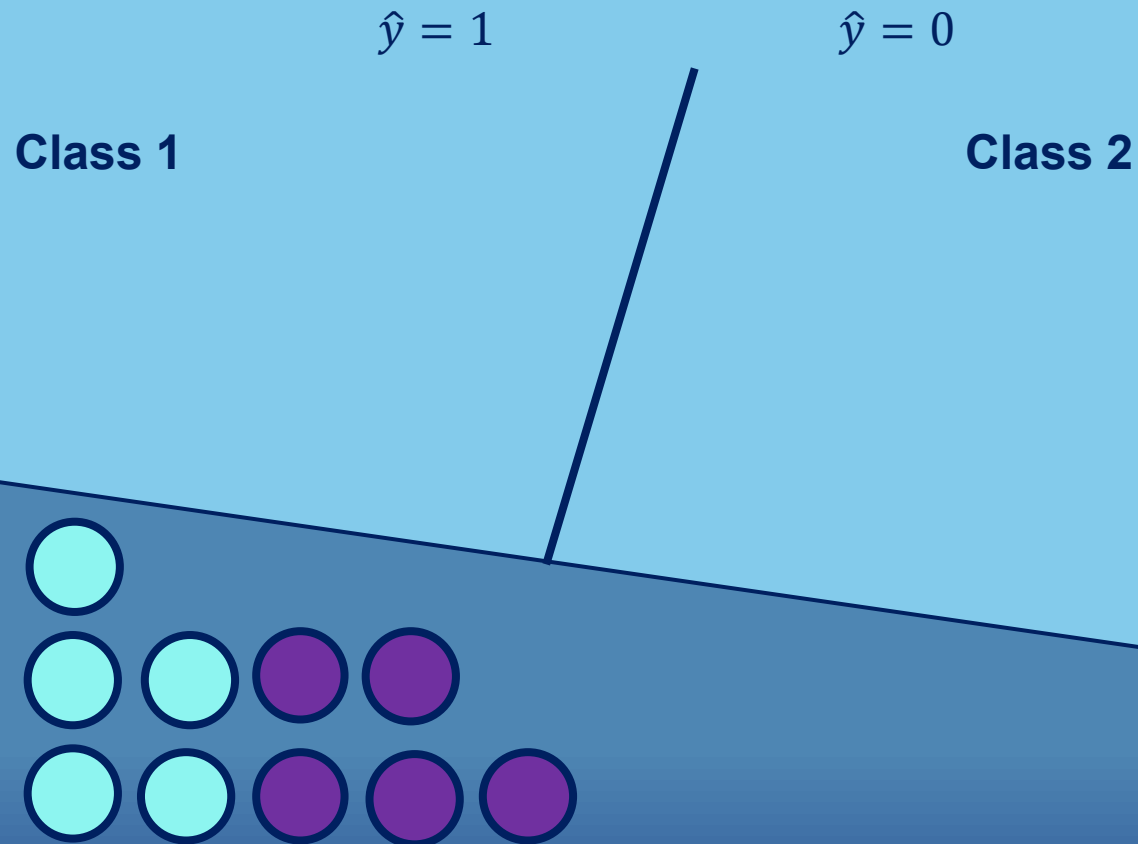
$$p(y|x; \theta) = \mathcal{N}(y; \hat{y}(x; \theta), \sigma^2 I)$$

Training Data: $\{(x_n, y_n)_{n=1}^N\}$ i.i.d.

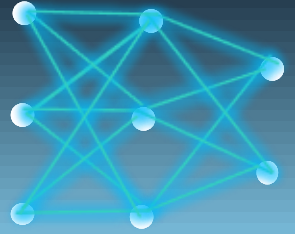
Sigmoid units for Bernoulli Distributions



- Consider **binary classification** such that $\hat{y} = 1$ denotes class C_1 and $\hat{y} = 0$ denotes class C_2



Sigmoid units for Bernoulli Distributions

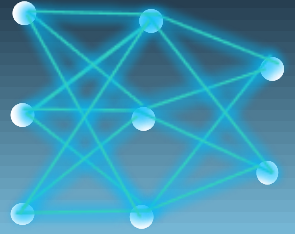


- A **Bernoulli distribution** is defined by just a **single number**. The neural network needs to predict only $p(y = 1|x)$
- For this number to be a **valid probability**, it must lie in the interval $[0,1]$
- Suppose we were to use a **linear unit**, and **threshold** its value **to obtain a valid probability**

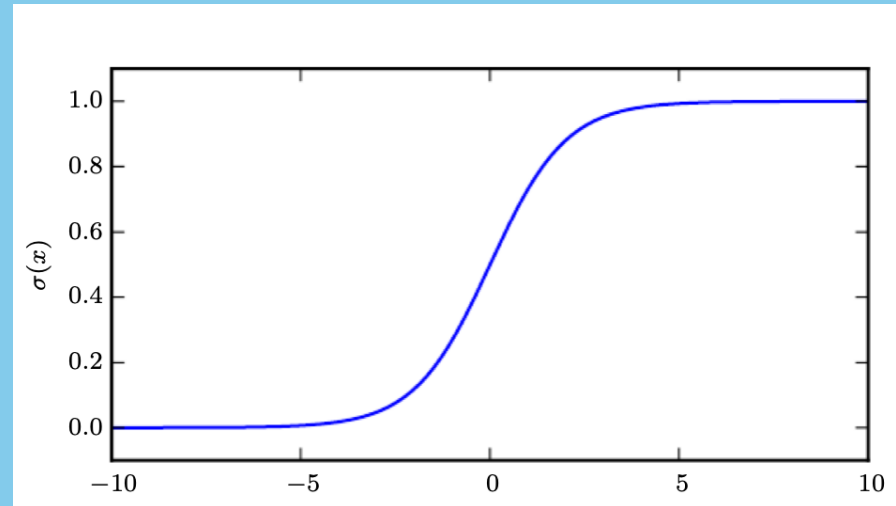
$$p(y = 1|x) = \max(0, \min(1, \mathbf{w}^T \mathbf{h} + b))$$

This would indeed define a **valid conditional distribution**, but we would **not be able to train it very effectively with gradient descent**

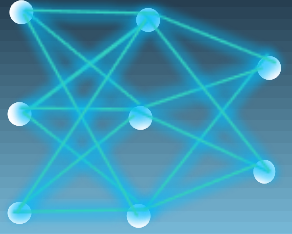
Sigmoid units for Bernoulli Distributions



- It is better to use a different approach that ensures **there is always a strong gradient whenever the model has the wrong answer**
- This approach is based on using **sigmoid output units combined with maximum likelihood**



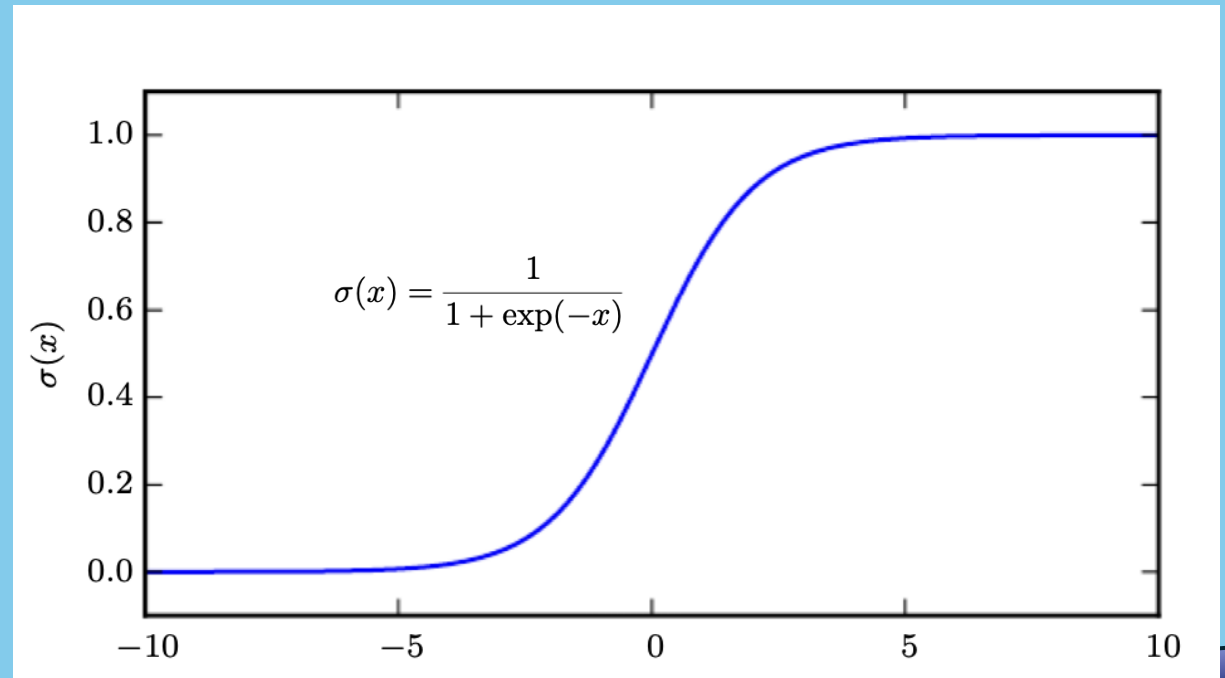
The Sigmoid function



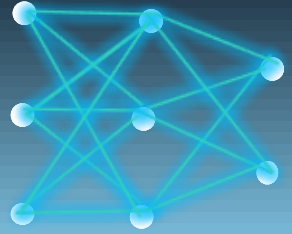
$$\sigma(x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}}$$

$$1 - \sigma(x) = 1 - \frac{e^x}{1 + e^x} = \frac{1}{1 + e^x} = \sigma(-x)$$

$$\begin{aligned} \frac{d\sigma(x)}{dx} &= \frac{d(1 + e^{-x})^{-1}}{dx} = -(1 + e^{-x})^{-2}(-e^{-x}) \\ &= \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} \\ &= \sigma(x)(1 - \sigma(x)) \end{aligned}$$



The Softplus function

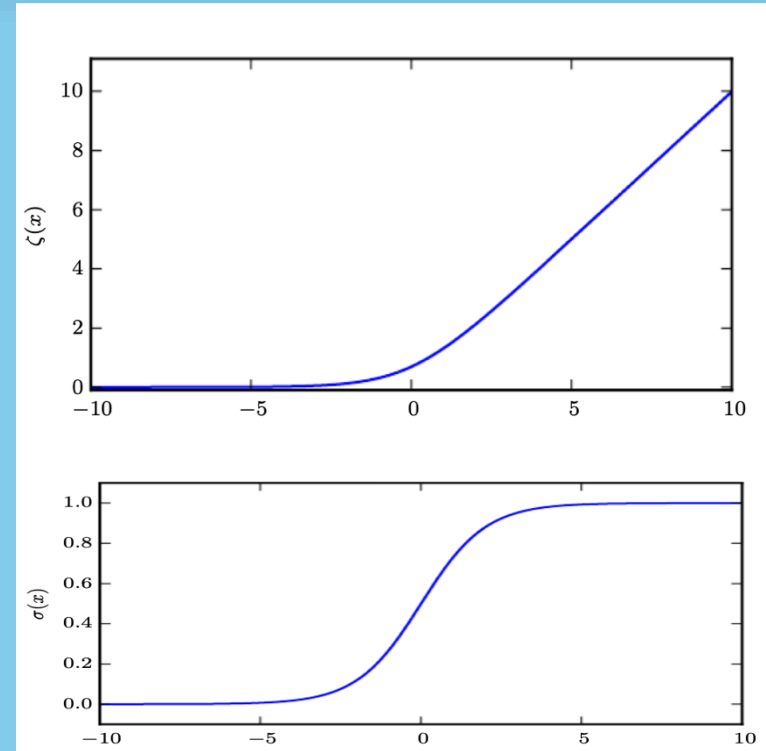


$$\zeta(x) = \log(1 + e^x)$$

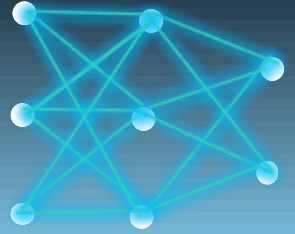
$$\zeta(x) = -\log((1 + e^x)^{-1})$$

$$\zeta(x) = -\log(\sigma(-x))$$

$$\frac{d\zeta(x)}{dx} = \frac{d\log(1 + e^x)}{dx} = \frac{e^x}{1 + e^x} = \sigma(x)$$



Sigmoid units for Bernoulli Distributions



- **This approach** to predicting the probabilities in log-space **is natural** to use with **maximum likelihood learning**
- Because the **cost function** used with maximum likelihood is $-\log(p(\mathbf{y}|\mathbf{x}; \boldsymbol{\theta}))$, the **log** in the **cost** function **undoes** the **exp** of the sigmoid
- **Without this effect**, the **saturation** of the **sigmoid** could **prevent** **gradient-based learning** from **making good progress**

Binary Classification

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sigma \left(\underbrace{\mathbf{w}^T \mathbf{h} + b}_z \right)$$

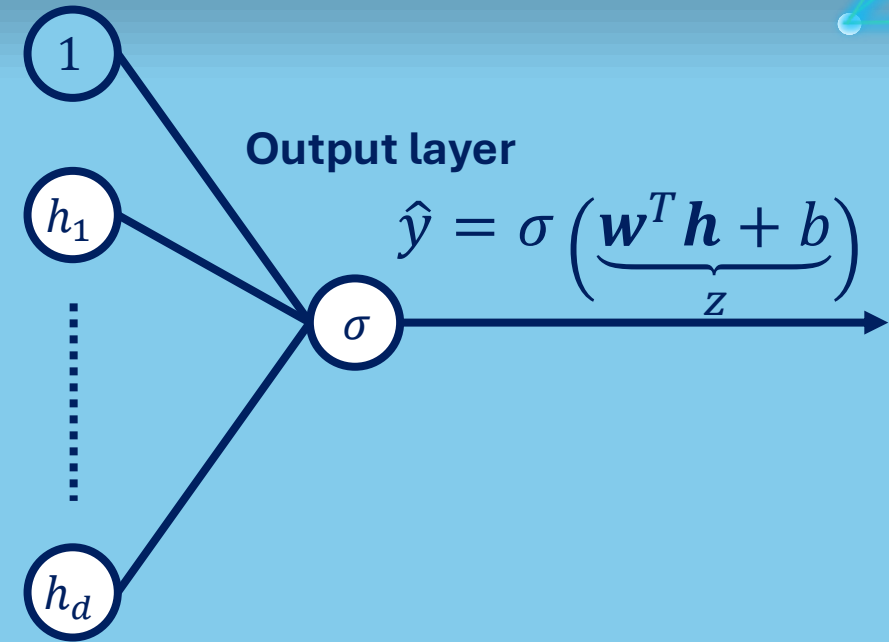
Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.

$$p(y_n | \mathbf{x}_n; \boldsymbol{\theta}) = f(\mathbf{x}_n; \boldsymbol{\theta})^{y_n} \cdot (1 - f(\mathbf{x}_n; \boldsymbol{\theta}))^{1-y_n}$$

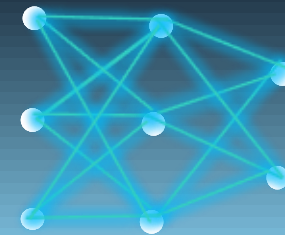
$$p(\mathbf{y} | \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{\theta}) = \prod_{n=1}^N p(y_n | \mathbf{x}_n; \boldsymbol{\theta})$$

$$p(\mathbf{y} | \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{\theta}) = \prod_{n=1}^N f(\mathbf{x}_n; \boldsymbol{\theta})^{y_n} \cdot (1 - f(\mathbf{x}_n; \boldsymbol{\theta}))^{1-y_n}$$

$$J(\boldsymbol{\theta}) = -\log \left(p(\mathbf{y} | \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{\theta}) \right) = - \sum_{n=1}^N y_n \log(f(\mathbf{x}_n; \boldsymbol{\theta})) + (1 - y_n) \log(1 - f(\mathbf{x}_n; \boldsymbol{\theta}))$$



Binary Classification



$$J(\theta) = - \sum_{n=1}^N y_n \log(\sigma(z_n)) + (1 - y_n) \log(1 - \sigma(z_n))$$

$$J(\theta) = - \sum_{n=1}^N y_n \log(\sigma(z_n)) + (1 - y_n) \log(\sigma(-z_n))$$

$$J(\theta) = - \sum_{n=1}^N \log(\sigma((2y_n - 1)z_n))$$

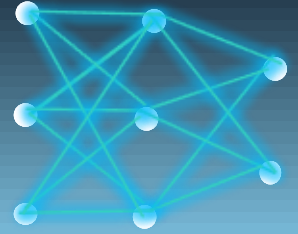
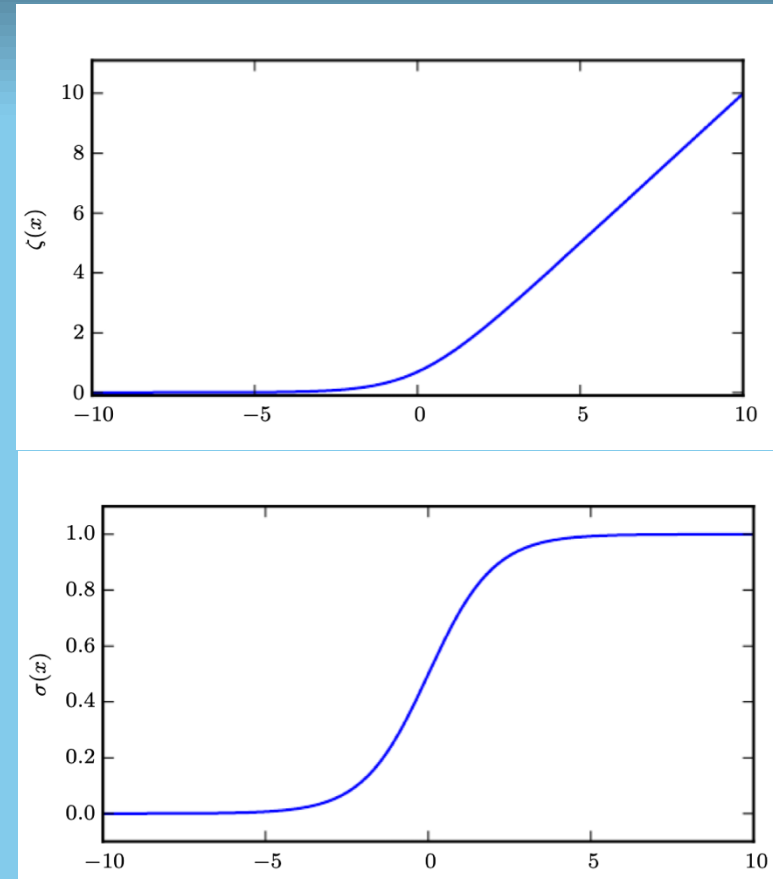
$$J(\theta) = \sum_{n=1}^N -\log(\sigma(-(1 - 2y_n)z_n))$$

$$J(\theta) = \sum_{n=1}^N \zeta((1 - 2y_n)z_n)$$

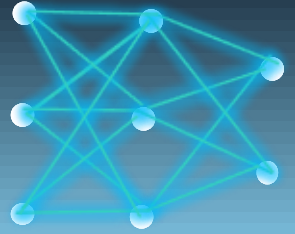
Binary Classification

$$J(\boldsymbol{\theta}) = \sum_{n=1}^N \zeta((1 - 2y_n)z_n)$$

- Saturation thus occurs only when the model already has the right answer
- Gradient-based learning can act to quickly correct a mistaken z

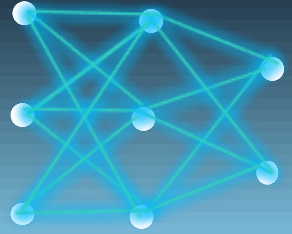


Sigmoid units for Bernoulli Distributions

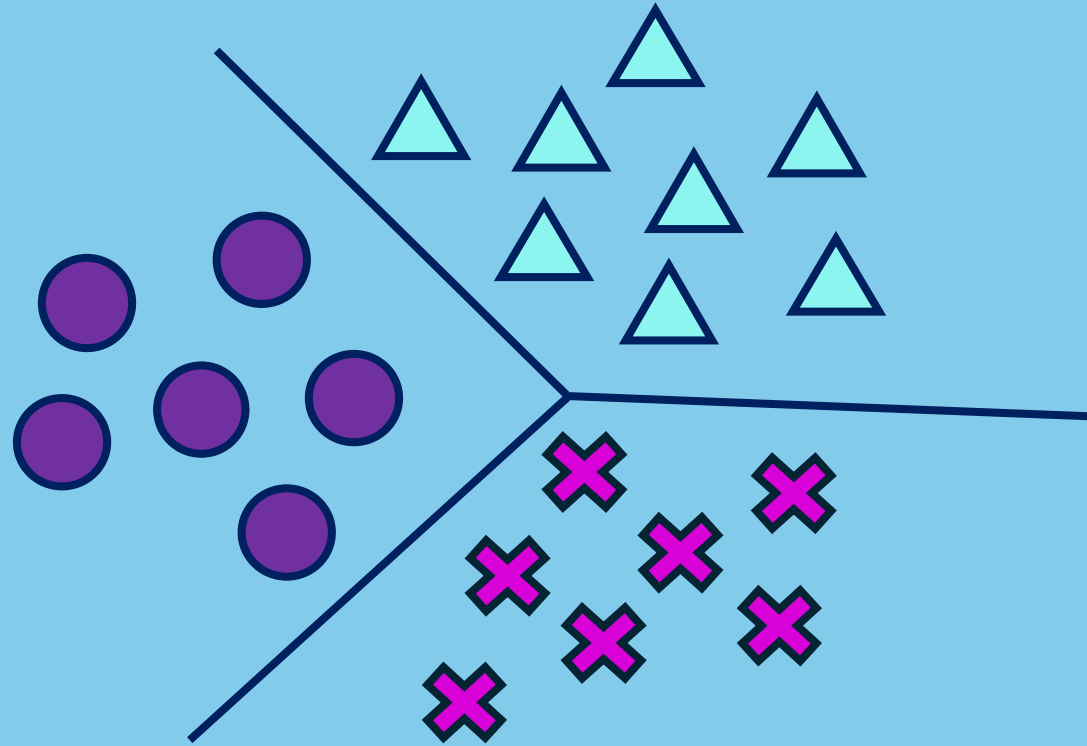


- The **sigmoid** activation function **saturates to 0** when **z becomes very negative** and **saturates to 1** when **z becomes very positive**
- The **gradient** can **shrink too small** to be useful for learning whenever this happens, whether the model has the correct answer or incorrect answer
- For this reason, **maximum likelihood** is **almost always** the **preferred** approach to **training sigmoid output units**
- When we use **other loss functions**, such as **mean squared error**, the **loss can saturate anytime $\sigma(z)$ saturates**

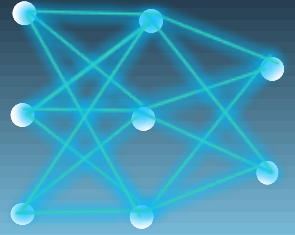
Multiclass Classification



Each input is assigned to one of K mutually exclusive classes



Softmax Units for Multinoulli Output distributions



- To **generalize** to the case of a **discrete variable with K values**, we now need to produce a vector $\hat{\mathbf{y}}$ with $\hat{y}_i = p(y = i|\mathbf{x})$
- We require that **each element of $\hat{\mathbf{y}}$ be between 0 and 1** and also that **the entire vector sums to 1** so that it **represents a valid probability distribution**

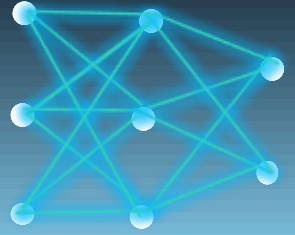
$$\mathbf{z} = \mathbf{W}^T \mathbf{h} + \mathbf{b}$$

$$z_i = \log(\tilde{p}(y = i|\mathbf{x}))$$

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

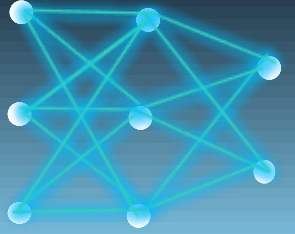
- Any time we wish to **represent a probability distribution** over a **discrete variable** with K **possible values**, we may use the **softmax function**

Saturation of the Softmax function



- Like the **sigmoid**, the **softmax** activation can **saturate**
- In the case of the **softmax**, there are **multiple output values**. These **output values** can **saturate** when the **differences between input values** become extreme
- When the **softmax** saturates, many **cost functions** based on the **softmax** also **saturate**, unless they are able to **invert the saturating activating function**
- **Many objective functions** other than the log-likelihood **do not work well** with the **softmax function**. In particular, **squared error** is a **poor loss function** for **softmax** units and **can fail to train** the model to change its output, even when the model makes highly confident incorrect predictions

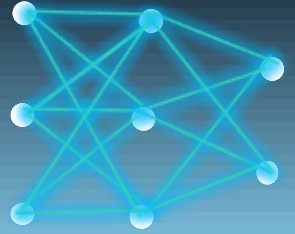
Saturation of the Softmax function



- The **softmax output** is **invariant** to **adding the same scalar** to all of its inputs

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Saturation of the Softmax function

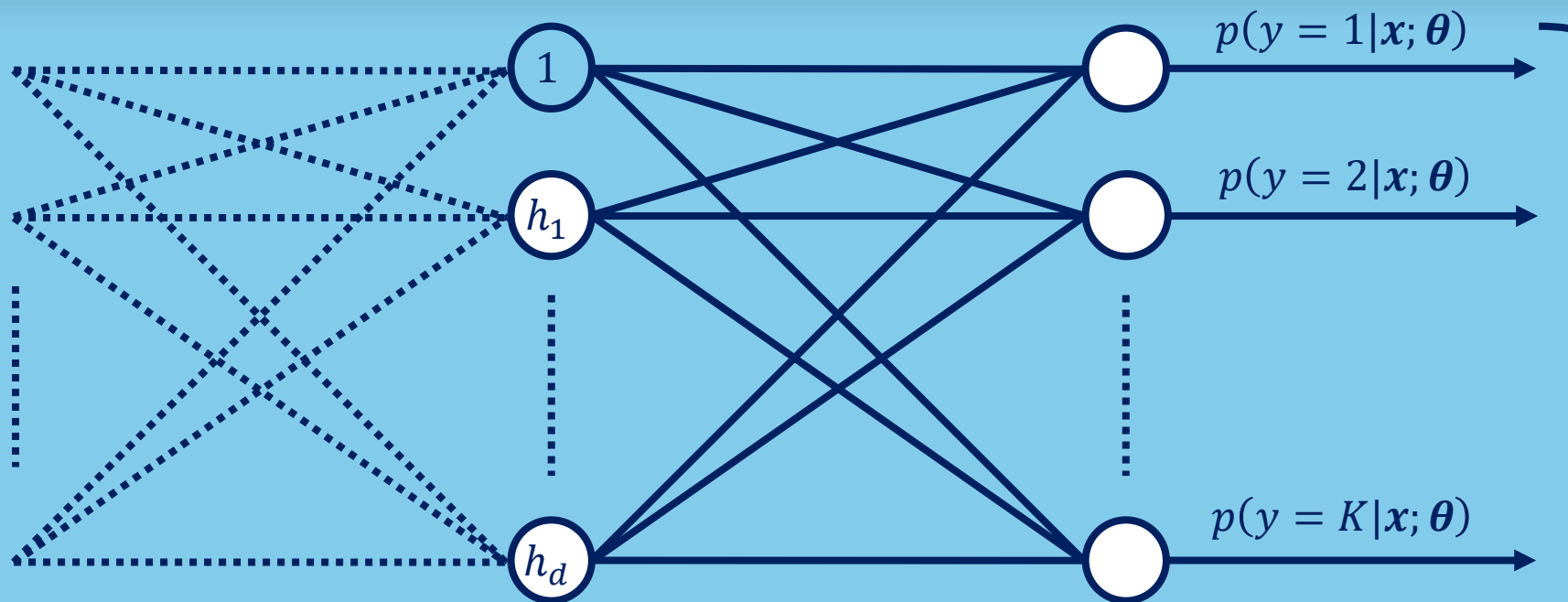
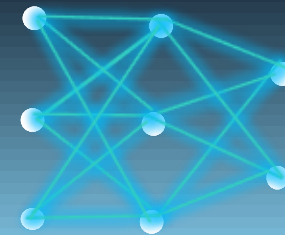


- The **softmax output** is **invariant** to **adding the same scalar** to all of its inputs

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i - c)}{\sum_j \exp(z_j - c)} \quad \text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - \max_j(z_j))$$

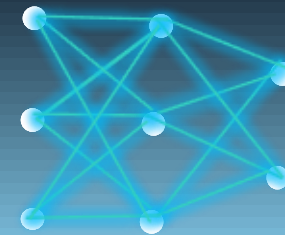
- An **output** $\text{softmax}(z)$ **saturates to 1** when the corresponding **input is maximal** ($z_i = \max_j(z_j)$) and z_i is **much greater** than **all the other inputs**. The **output** can also **saturate to 0** when z_i is not maximal and **the maximum is much greater**

Multiclass Classification



$$\mathbf{z} = \mathbf{W}^T \mathbf{h} + \mathbf{b}$$
$$z_i = \log(\tilde{p}(y = i | \mathbf{x}; \boldsymbol{\theta}))$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$
$$\hat{\mathbf{y}} = f(\mathbf{x}; \boldsymbol{\theta}) = \begin{bmatrix} p(y = 1 | \mathbf{x}; \boldsymbol{\theta}) \\ \vdots \\ p(y = K | \mathbf{x}; \boldsymbol{\theta}) \end{bmatrix}$$
$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$



Thank you